EXECUTION BLUEPRINT V1.0

# MemoryVault AI: Development Plan

End-to-End Implementation Roadmap, Milestones & Alternative Stacks

## 1. Project Overview & MVP Scope

**MemoryVault AI** is an intelligent chatbot predicting misplaced item locations using contextual habit modeling and outputting physical search routes. The MVP is restricted to a single-user web app, pre-trained on ~5,000 synthetic records, executing A* search on a static graph.

## 2. Definition of Done (DoD)

- **Feature Completeness:** Core flow (Chat → Inference → A* Route → Feedback Log) executes without fatal errors.
- **ML Metrics:** Random Forest achieves ≥ 80% Top-3 accuracy; ROC-AUC ≥ 0.85 on test splits.
- **Performance:** Chatbot turnaround latency < 2.0 seconds.
- **Quality Assurance:** 80% unit test coverage on API routes; verified cross-browser UI responsiveness.

## 3. Alternative Tech Stack Suggestions

| Domain | Planned MVP Stack | Alternative / Scaled Stack Recommendations |
|---|---|---|
| **Backend API** | Python / Flask | **FastAPI:** Offers native async and Pydantic validation (ideal for ML payloads). |
| **Frontend UI** | Vanilla JS / HTML / CSS | **Next.js / React:** Better component state management for chat interfaces. |
| **Machine Learning** | scikit-learn (Random Forest) | **XGBoost / LightGBM:** Faster inference and better handling of tabular context data. |
| **Database** | SQLite | **PostgreSQL / Supabase:** For multi-user scaling and cloud deployment. |
| **AI Dev Tools** | Standard IDE | **Cursor / GitHub Copilot:** Accelerate Flask boilerplate and A* pathfinding generation. |

## 4. Phased Development Roadmap

Following the principle that each phase must be independently testable and avoid spending too much time on infrastructure nobody can interact with yet:

### Phase 1: Environment Setup & Foundation

DURATION: 1 WEEK

- Initialize Git repository, Python virtual environment, and dependency `requirements.txt`.
- Scaffold Flask application structure (routes, blueprints, error handlers).
- Initialize SQLite database schema for `session_logs` and `user_feedback`.
- Set up basic API health check endpoint (`/api/health`).

### Phase 2: Data Synthesis & ML Pipeline

DURATION: 1.5 WEEKS

- Write `generate_dataset.py` to create ~5,000 context records (time, weather, fatigue, item, etc.).
- Implement Unsupervised Pipeline: Apply PCA for dimensionality reduction and K-Means for habit clustering.
- Implement Supervised Pipeline: Train Random Forest classifier, evaluate via 5-fold CV, and serialize artifacts (`.joblib`).
- Expose `/api/v1/analytics/metrics` endpoint to retrieve confusion matrices.

### Phase 3: Spatial Graph & Routing Logic

**DURATION: 1 WEEK**

- Define static JSON residential topology (nodes, adjacencies, distances).
- Implement A* Search Algorithm dynamically weighing physical distance against ML probability scores.
- Create isolated unit tests validating that the A* router prunes 0% probability nodes.

### Phase 4: Backend Integration & Conversational Controller

**DURATION: 1 WEEK**

- Build the `/api/v1/chat` endpoint connecting input parsing → ML Inference → A* Router.
- Implement Dialogue State Manager to track missing context (e.g., asking for weather if missing).
- Validate JSON schemas using Pydantic/Marshmallow to reject malformed UI requests.

### Phase 5: Frontend UI & UX Polish

**DURATION: 1.5 WEEKS**

- Build responsive split-pane layout (Chat on left, Route on right).
- Wire Vanilla JS fetch calls to the Flask API.
- Implement contextual quick-reply chips for 1-tap user responses.
- Build interactive Route Cards with "Found Here" buttons logging to `/api/v1/feedback`.

### Phase 6: Deployment & Monitoring

**DURATION: 3 DAYS**

- Write multi-stage Dockerfile using `python:3.10-slim` and Gunicorn.
- Deploy to a free/low-cost tier (e.g., Render, Railway, or local Docker instance).
- Conduct end-to-end user acceptance testing (UAT).

## 5. Critical Priorities & Dependencies

- **Priority 1:** The ML Pipeline (Phase 2) is the critical path. The A* router and Flask API cannot be validated without real probability arrays.
- **Dependency:** Frontend Development (Phase 5) is strictly dependent on the completion of the `/api/v1/chat` contract in Phase 4. Mock API responses should be used if UI work begins early.